Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

Week 3 Exercise

Objective 1: Conceptual Foundations

In the late 2000s (around 2007–2008), Facebook faced a major analytics challenge: engineers needed to query massive datasets stored in Hadoop, but most analysts only knew SQL. At the time, working directly with Hadoop required writing Java-based MapReduce jobs, which created a steep barrier for non-engineers. Hive was created to solve this problem by providing a SQL-like interface that allows users to query large-scale data without writing Java code. Users can interact with Hive through a command-line interface or connect it to BI tools such as Tableau or Power BI, making large-scale data analysis accessible to analysts and data scientists who are already familiar with SQL.

Hive operates through a layered architecture. When a user submits a SQL query, the Hive driver first parses it and converts it into an abstract syntax tree. The optimizer then analyzes the query and determines an efficient execution plan. That plan is passed to the execution engine, which translates the query into distributed jobs. Historically, these jobs ran as MapReduce tasks managed by YARN, but modern Hive commonly uses engines like Tez or Spark for better performance. The actual data is stored in Hadoop Distributed File System (HDFS), and the execution engine processes that data in parallel across the cluster.

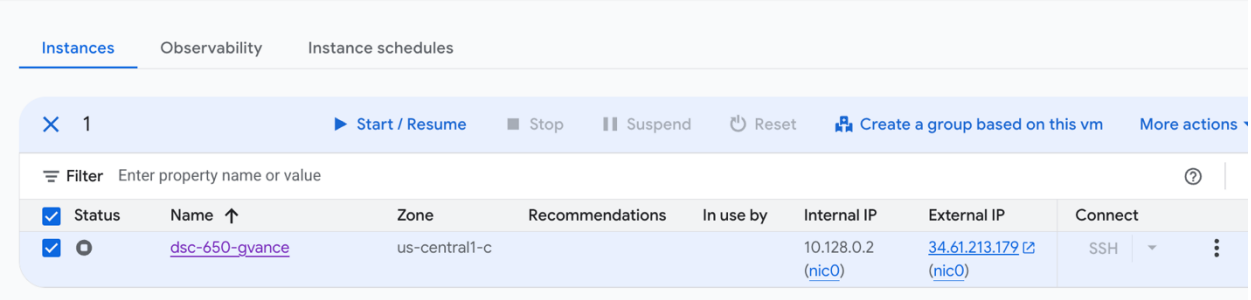
Hive differs from a traditional DBMS because it is not a database itself—it is a SQL translation and execution layer that sits on top of distributed storage. Its role is similar to modern "serverless" query services such as AWS Athena, Azure Synapse (serverless SQL), and Google

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

BigQuery, which also allow SQL queries over large datasets without managing infrastructure. Hive relies on a metastore to store metadata such as table schemas, column types, and data locations, while the raw data lives in HDFS. When a managed Hive table is dropped, both the metadata and the underlying HDFS data are deleted; external tables are an exception, as dropping them removes only the metadata and leaves the data intact. Hive also supports both ACID and non-ACID tables: ACID tables provide transactional guarantees using formats like ORC and additional metadata, while non-ACID tables are simpler and optimized for read-heavy or external data use cases.

Objective 2 – Resize Google VM

To edit the VM, we'll first stop it using Google Console. Changing the resources will allow the VM to process higher volumes of data. We'll be sure to adjust the VM only after powered down. While these big data tools will process data with more efficacy, we will still need to add more CPUs/RAM.



Objective 3: Hive Managed Tables

We'll run the query (a basic select *) to confirm that the table was created and we can access the data inside hive.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
Time taken: 1.066 seconds
hive> SELECT * FROM grades;
OK
Alfalfa Aloysius        123-45-6789     40.0    90      100.0   83.0    49.0    D-
Alfred  University      123-12-1234     41.0    97      96.0    97.0    48.0    D+
Gerty   Gramma  567-89-0123     41.0    80      60.0    40.0    44.0    C
Android Electric        087-65-4321     42.0    23      36.0    45.0    47.0    B-
Bumpkin Fred    456-78-9012     43.0    78      88.0    77.0    45.0    A-
Rubble  Betty   234-56-7890     44.0    90      80.0    90.0    46.0    C-
Noshow  Cecil   345-67-8901     45.0    11      -1.0    4.0     43.0    F
Buff    Bif     632-79-9939     46.0    20      30.0    40.0    50.0    B+
Airpump Andrew  223-45-6789     49.0    1       90.0    100.0   83.0    A
Backus  Jim     143-12-1234     48.0    1       97.0    96.0    97.0    A+
Carnivore       Art     565-89-0123     44.0    1       80.0    60.0    40.0    D+
Dandy   Jim     087-75-4321     47.0    1       23.0    36.0    45.0    C+
Elephant        Ima     456-71-9012     45.0    1       78.0    88.0    77.0    B-
Franklin        Benny   234-56-2890     50.0    1       90.0    80.0    90.0    B-
George  Boy     345-67-3901     40.0    1       11.0    -1.0    4.0     B
Heffalump       Harvey  632-79-9439     30.0    1       20.0    30.0    40.0    C
Time taken: 2.446 seconds, Fetched: 16 row(s)
hive>
```

First query is a basic select *, but orders by description. Because we pass in the 8th column, we sort without requiring the name.

SELECT * FROM grades ORDER BY 8 DESC; This sorts by the 8th column

```
hive> SELECT * FROM grades ORDER BY 8 DESC;
2026-01-09 18:08:13,993 INFO  [41555300-801f-481e-b56e-e04de75d1a0d main] reducesink.VectorReduceSinkObjectHashOperator: VectorReduceSinkObjectHashOperator cons
tructor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorReduceSinkInfo@91a1005
Query ID = root_20260109180813_a9681abb-52c2-420f-a273-82488b709a17
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1767981469663_0002)

----------------------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
----------------------------------------------------------------------------------------------
Map 1 ..... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ..... container     SUCCEEDED      1          1        0        0       0       0
----------------------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 5.15 s
----------------------------------------------------------------------------------------------
OK
Backus  Jim     143-12-1234     48.0    1       97.0    96.0    97.0    A+
Franklin        Benny   234-56-2890     50.0    1       90.0    80.0    90.0    B-
Airpump Andrew  223-45-6789     49.0    1       90.0    100.0   83.0    A
Elephant        Ima     456-71-9012     45.0    1       78.0    88.0    77.0    B-
Buff    Bif     632-79-9939     46.0    20      30.0    40.0    50.0    B+
Alfalfa Aloysius        123-45-6789     40.0    90      100.0   83.0    49.0    D-
Alfred  University      123-12-1234     41.0    97      96.0    97.0    48.0    D+
Android Electric        087-65-4321     42.0    23      36.0    45.0    47.0    B-
Rubble  Betty   234-56-7890     44.0    90      80.0    90.0    46.0    C-
Bumpkin Fred    456-78-9012     43.0    78      88.0    77.0    45.0    A-
Dandy   Jim     087-75-4321     47.0    1       23.0    36.0    45.0    C+
Gerty   Gramma  567-89-0123     41.0    80      60.0    40.0    44.0    C
Noshow  Cecil   345-67-8901     45.0    11      -1.0    4.0     43.0    F
Carnivore       Art     565-89-0123     44.0    1       80.0    60.0    40.0    D+
Heffalump       Harvey  632-79-9439     30.0    1       20.0    30.0    40.0    C
George  Boy     345-67-3901     40.0    1       11.0    -1.0    4.0     B
Time taken: 6.036 seconds, Fetched: 16 row(s)
hive>
```

Next query assigns an alias for the average of test scores.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
SELECT AVG(test1) AS avg_final_score FROM grades;
```

```
hive> SELECT AVG(test1) AS avg_final_score FROM grades
    > ;
2026-01-09 18:10:23,501 INFO  [41555300-801f-481e-b56e-e04de75d1a0d main] reducesink.VectorReduceSinkEmptyKeyOperator: VectorReduceSinkEmptyKeyOperator construc
tor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorReduceSinkInfo@634d3439
Query ID = root_20260109181023_a0f48afe-e9c3-4240-bff3-82165b0e4e2c
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1767981469663_0002)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ...... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 4.77 s
--------------------------------------------------------------------------------
OK
43.4375
Time taken: 6.002 seconds, Fetched: 1 row(s)
hive>
```

The third and final query counts the students per grade so we can group/bin the dataset.

SELECT final, COUNT(*) FROM grades GROUP BY final;

```
hive> SELECT final, COUNT(*) FROM grades GROUP BY final;
Query ID = root_20260109181222_471ec21e-286f-4ead-8f30-25142429f3f3
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1767981469663_0002)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ...... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 5.00 s
--------------------------------------------------------------------------------
OK
4.0     1
40.0    2
43.0    1
44.0    1
45.0    2
46.0    1
47.0    1
48.0    1
49.0    1
50.0    1
77.0    1
83.0    1
90.0    1
97.0    1
Time taken: 5.815 seconds, Fetched: 14 row(s)
hive>
```

Objective 4: Hive with a new Dataset

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

The dataset is from Kaggle. It was loaded into github and then downloaded using the wget command. Once inside the container, the csv was moved into the HDFS, and it was accessible to Hive. However, we first needed a table to perform queries against.

```
hive> DROP TABLE soil_nutrient_pools;
OK
Time taken: 0.294 seconds
hive>
    > CREATE EXTERNAL TABLE soil_nutrient_pools (
    >     SampleID STRING,
    >     site STRING,
    >     block INT,
    >     paddock INT,
    >     plot INT,
    >     slope INT,
    >     rainfall_reduction INT,
    >     grazing_treatment STRING,
    >     year INT,
    >     type STRING,
    >     faith_pd STRING
    > )
    > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
    > WITH SERDEPROPERTIES (
    >   "separatorChar" = ",",
    >   "quoteChar"     = "\"",
    >   "escapeChar"    = "\\"
    > )
    > STORED AS TEXTFILE
    > LOCATION '/';
OK
Time taken: 0.29 seconds
hive>
```

First we do a basic query * count to see if Hive can read the rows. This is important because the first time I made a table, I had to drop it because while the table creation command succeeded, the location was wrong, which led to returning 0 for the counts. Note that this command took 37 seconds to load because we had a large number of rows.

```
hive> select COUNT(*) from soil_nutrient_pools
    > ;
2026-01-09 20:02:30,175 INFO  [d1f1ae82-55e5-4030-84c6-86ec145532dc main] reducesink.VectorReduceSinkEmptyKeyOperat
or: VectorReduceSinkEmptyKeyOperator constructor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorReduceSi
nkInfo@3b780b5c
Query ID = root_20260109200229_ce7642c7-2195-4f29-b5d7-2245a8c63553
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
2026-01-09 20:02:30,378 INFO  [d1f1ae82-55e5-4030-84c6-86ec145532dc main] client.RMProxy: Connecting to ResourceMan
ager at master/172.28.1.1:8032
Session re-established.
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1767981469663_0019)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ...... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 5.03 s
--------------------------------------------------------------------------------
OK
1006
Time taken: 11.47 seconds, Fetched: 1 row(s)
hive>
```

The next query selected certain columns, such as site, year, and type. Limit results to 5.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
hive> SELECT SampleID, site, year, type, faith_pd
    > FROM soil_nutrient_pools
    > LIMIT 5;
OK
GMDR-FK-2018-4  FK      2018    bacteria        3.912.908.739
GMDR-TB-2018-45 TB      2018    bacteria        3.298.556.468
GMDR-TB-2018-8  TB      2018    bacteria        3.332.257.877
GMDR-TB-2018-31 TB      2018    bacteria        3.806.342.641
GMDR-TB-2018-23 TB      2018    bacteria        4.135.953.774
Time taken: 0.182 seconds, Fetched: 5 row(s)
hive>
```

The 3rd query grouped by site, which turned out to only be 3 groups total.

```
Time taken: 11.47 seconds, Fetched: 1 row(s)
hive> SELECT
    >   site,
    >   COUNT(*) AS n_rows
    > FROM soil_nutrient_pools
    > GROUP BY site
    > ORDER BY site;
2026-01-09 20:05:19,448 INFO  [d1f1ae82-55e5-4030-84c6-86ec145532dc main] reducesink.VectorReduceSinkObjectHashOper
ator: VectorReduceSinkObjectHashOperator constructor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorRedu
ceSinkInfo@3e4636c3
Query ID = root_20260109200519_516710f9-ec12-401d-a0e8-046ed70e3bbe
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1767981469663_0019)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ...... container     SUCCEEDED      1          1        0        0       0       0
Reducer 3 ...... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 03/03  [==========================>>] 100%  ELAPSED TIME: 4.96 s
--------------------------------------------------------------------------------
OK
FK      520
TB      486
Time taken: 5.872 seconds, Fetched: 2 row(s)
hive>
```

Now we will run explain with each SQL to extract the stages of each query.

• **How many stages are involved.**
o **2 stages** – a fetch stage and a distributed execution stage using Map → Reduce.

• **Whether the query sorts or aggregates data.**
o **It aggregates data** using a global `COUNT(*)`.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

• **What the shuffle process is doing.**
o The **partition-only shuffle** sends partial counts from each mapper, which processes a chunk of the table, to the reducer where they are combined.

• **How Hive is translating your SQL into distributed execution.**
o Hive splits the table into chunks, processes them in parallel on multiple machines, and reduces the partial results into a single final count.

```
    > EXPLAIN SELECT COUNT(*)
    > FROM soil_nutrient_pools;
2026-01-10 18:10:34,621 INFO  [c3cb3fb4-68f3-489a-9288-38f36f6fdcf2 main] reducesink.VectorReduceSinkEmptyKeyOperat
or: VectorReduceSinkEmptyKeyOperator constructor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorReduceSi
nkInfo@70aa58e1
OK
Plan optimized by CBO.

Vertex dependency in root stage
Reducer 2 <- Map 1 (CUSTOM_SIMPLE_EDGE)

Stage-0
  Fetch Operator
    limit:-1
    Stage-1
      Reducer 2 vectorized
      File Output Operator [FS_11]
        Group By Operator [GBY_10] (rows=1 width=8)
          Output:["_col0"],aggregations:["count(VALUE._col0)"]
        <-Map 1 [CUSTOM_SIMPLE_EDGE] vectorized
          PARTITION_ONLY_SHUFFLE [RS_9]
            Group By Operator [GBY_8] (rows=1 width=8)
              Output:["_col0"],aggregations:["count()"]
              Select Operator [SEL_7] (rows=920 width=708)
                TableScan [TS_0] (rows=920 width=708)
                  default@soil_nutrient_pools,soil_nutrient_pools,Tbl:COMPLETE,Col:COMPLETE

Time taken: 4.75 seconds, Fetched: 21 row(s)
hive>
```

• **How many stages are involved.**
o **1 stage** – a fetch stage only. Hive scans the table and returns rows directly. There is no map-reduce stage.

• **Whether the query sorts or aggregates data.**
o **It does not sort or aggregate**. There is no `ORDER BY` or aggregation function; it simply reads rows.

• **What the shuffle process is doing.**
o **No shuffle occurs**. Since there is no aggregation or grouping, Hive does not move data between machines.

• **How Hive is translating your SQL into distributed execution.**
o Hive scans the table, selects the requested columns, stops after retrieving 5 rows due to the `LIMIT`, and returns the result without performing distributed aggregation.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
Time taken: 4.75 seconds, Fetched: 21 row(s)
hive> EXPLAIN SELECT
    >     SampleID,
    >     site,
    >     year,
    >     type,
    >     faith_pd
    > FROM soil_nutrient_pools
    > LIMIT 5;
OK
Plan optimized by CBO.

Stage-0
  Fetch Operator
    limit:5
    Limit [LIM_2]
      Number of rows:5
      Select Operator [SEL_1]
        Output:["_col0","_col1","_col2","_col3","_col4"]
        TableScan [TS_0]
          Output:["sampleid","site","year","type","faith_pd"]

Time taken: 0.177 seconds, Fetched: 12 row(s)
hive>
```

• **How many stages are involved.**
o **1 stage** – a fetch stage only. Hive scans the table and returns rows directly.

• **Whether the query sorts or aggregates data.**
o **It neither sorts nor aggregates data.** The query simply selects columns and limits the output.

• **What the shuffle process is doing.**
o **No shuffle occurs.** Since there is no aggregation or grouping, Hive does not move data between machines.

• **How Hive is translating your SQL into distributed execution.**
o Hive reads the table, selects the specified columns, stops after retrieving 5 rows due to the `LIMIT`, and returns the results without any distributed reduction.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
hive> EXPLAIN SELECT
    >     site,
    >     COUNT(*) AS n_rows
    > FROM soil_nutrient_pools
    > GROUP BY site
    > ORDER BY site;
2026-01-10 18:11:54,427 INFO  [c3cb3fb4-68f3-489a-9288-38f36f6fdcf2 main] reducesink.VectorReduceSinkObjectHashOper
ator: VectorReduceSinkObjectHashOperator constructor vectorReduceSinkInfo org.apache.hadoop.hive.ql.plan.VectorRedu
ceSinkInfo@577cf459
OK
Plan optimized by CBO.

Vertex dependency in root stage
Reducer 2 <- Map 1 (SIMPLE_EDGE)
Reducer 3 <- Reducer 2 (SIMPLE_EDGE)

Stage-0
  Fetch Operator
    limit:-1
    Stage-1
      Reducer 3 vectorized
      File Output Operator [FS_15]
        Select Operator [SEL_14] (rows=460 width=708)
          Output:["_col0","_col1"]
        <-Reducer 2 [SIMPLE_EDGE] vectorized
          SHUFFLE [RS_13]
            Group By Operator [GBY_12] (rows=460 width=708)
              Output:["_col0","_col1"],aggregations:["count(VALUE._col0)"],keys:KEY._col0
            <-Map 1 [SIMPLE_EDGE] vectorized
              SHUFFLE [RS_11]
                PartitionCols:_col0
                Group By Operator [GBY_10] (rows=920 width=708)
                  Output:["_col0","_col1"],aggregations:["count()"],keys:site
                  Select Operator [SEL_9] (rows=920 width=708)
                    Output:["site"]
                    TableScan [TS_0] (rows=920 width=708)
                      default@soil_nutrient_pools,soil_nutrient_pools,Tbl:COMPLETE,Col:NONE,Output:["site"]

Time taken: 0.194 seconds, Fetched: 28 row(s)
hive>
```

Objective 5: Hive External Tables

In the last step we're going to add an external table to demonstrate the schema-on-read-capability. First we'll add the table along with a basic query to verify the results are present.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
hive>     CREATE EXTERNAL TABLE grades_ext(
    >     `Last name` STRING,
    >     `First name` STRING,
    >     `SSN` STRING,
    >     `Test1` DOUBLE,
    >     `Test2` INT,
    >     `Test3` DOUBLE,
    >     `Test4` DOUBLE,
    >     `Final` DOUBLE,
    >     `Grade` STRING
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE
    > LOCATION '/rawdata'
    > TBLPROPERTIES ("skip.header.line.count"="1");
OK
Time taken: 1.336 seconds
hive> SELECT * FROM grades_ext;
OK
Alfalfa Aloysius        123-45-6789     40.0    90      100.0   83.0    49.0    D-
Alfred  University      123-12-1234     41.0    97      96.0    97.0    48.0    D+
Gerty   Gramma  567-89-0123     41.0    80      60.0    40.0    44.0    C
Android Electric        087-65-4321     42.0    23      36.0    45.0    47.0    B-
Bumpkin Fred    456-78-9012     43.0    78      88.0    77.0    45.0    A-
Rubble  Betty   234-56-7890     44.0    90      80.0    90.0    46.0    C-
Noshow  Cecil   345-67-8901     45.0    11      -1.0    4.0     43.0    F
Buff    Bif     632-79-9939     46.0    20      30.0    40.0    50.0    B+
Airpump Andrew  223-45-6789     49.0    1       90.0    100.0   83.0    A
Backus  Jim     143-12-1234     48.0    1       97.0    96.0    97.0    A+
Carnivore       Art     565-89-0123     44.0    1       80.0    60.0    40.0    D+
Dandy   Jim     087-75-4321     47.0    1       23.0    36.0    45.0    C+
Elephant        Ima     456-71-9012     45.0    1       78.0    88.0    77.0    B-
Franklin        Benny   234-56-2890     50.0    1       90.0    80.0    90.0    B-
George  Boy     345-67-3901     40.0    1       11.0    -1.0    4.0     B
Heffalump       Harvey  632-79-9439     30.0    1       20.0    30.0    40.0    C
Time taken: 2.307 seconds, Fetched: 16 row(s)
hive>
```

Next, we'll drop the table inside Hive. This is completely okay because we will not be losing the original data!

```
hive>  DROP TABLE grades_ext;
OK
Time taken: 0.171 seconds
hive> SHOW TABLES;
OK
grades
soil_nutrient_pools
Time taken: 0.088 seconds, Fetched: 2 row(s)
hive>
```

Dropping the external table removes only Hive's metadata. The underlying CSV file is not deleted, so it remains accessible on the VM's filesystem.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 9, 2026

```
bash-5.0# hdfs dfs -ls /rawdata
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org
/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLogge
rBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticL
oggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-01-10 18:33:08,931 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.
.. using builtin-java classes where applicable
Found 1 items
-rw-r--r--   1 root supergroup        747 2026-01-10 18:14 /rawdata/grades.csv
bash-5.0#
```